Method Signature

~~Return type~~ + method name + Parameter list

?

method signature

Sum(5,4)

Sum(5,4,8)

Method overloading:

Sum (int x, int y)

Sum (int x, int y, int z)

Sum (int x, double y)

Sum (double x, int y)

~~Sum~~ (int x, int y)

~~Sum~~ (int a, int b)

Sum(5,4)

main { int x = Sum(5,4)

double y = Sum(5,4)

}

~~int~~ Sum (int x, int y)

double Sum (int x, int y)

Call by value

Public class test

{ Public static void main(String[] args)

{ int x=3, y=6, z=8;

Sy —— (getlarger(x, z));

Sy —— (getlarger(y, z, x));

}

Public Static int getlarger (int x, int y)

{ if (x > y)

return x;

else

return y;

}

Public Static int getlarger (int x, int y, int z)

{ if (x > y && x > z)

return x;

else if (y > x && y > z)

return y;

else

return z;

}

8

8

Call by reference:

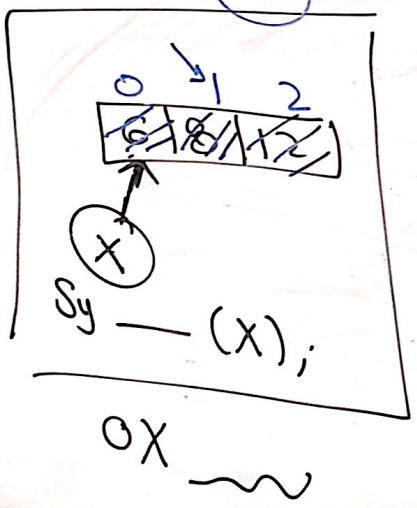
address

int[] x = {6, 8, 12}

array عنوان

x + 0 → x[0]

x[1]




```
main
{
  int x = 5;
  Sy — ("x=" + x);
  increase(x);
  Sy — ("x=" + x);
}
Public static void increase(int x)
{
  x = x * 2;
}
```

x=5
x=5

5

5

x = 10

```
int x = 5;
Sy — (x);
int[] x = {5,4,3};
— (x);
```

5

address

Call by reference:

```
import java.util.Arrays;
Public static void main(String[] args)
{
  int[] x = {5,3,8};
  Sy — (Arrays.toString(x));
  increase(x);
  Sy — (Arrays.toString(x));
}
Public static void increase(int[] y)
{
  for (int i=0; i < y.length; y++)
  {
    y[i] = y[i] * 2;
  }
}
```

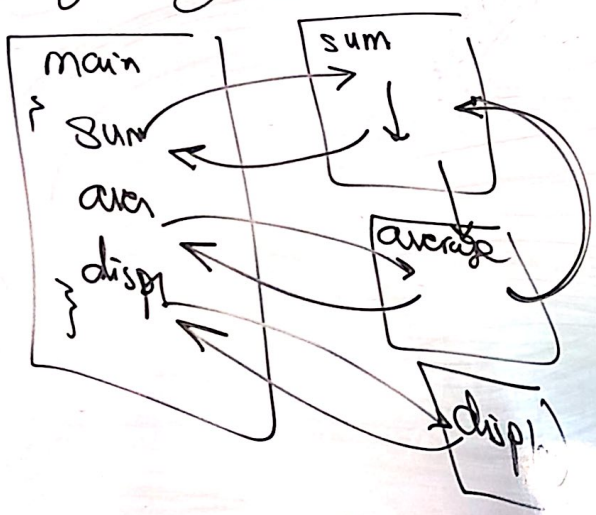
[5,3,8]
[10,6,16]

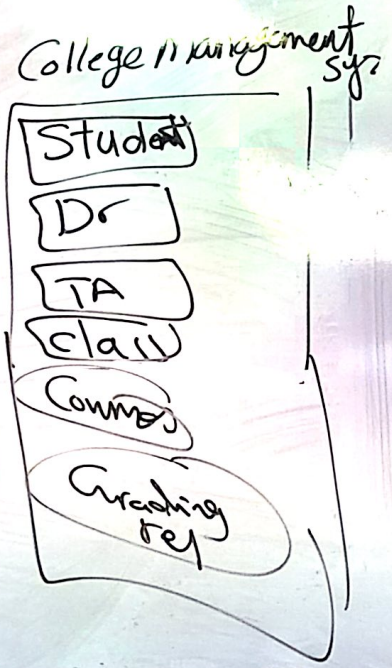
Memory

object oriented Programming (oop)

. It is a Programming Paradigm based on the Concept of objects

Programming Paradigm:





object
- Data
- operations (methods)

Set get

Rectangle
- Data
  - Length
  - Width
- operation
  - Change length
  - Change
  - get